UNIVERSIDAD DE SAN CARLOS DE GUATEMALA
CENTRO UNIVERSITARIO DE OCCIDENTE
DIVISIÓN DE CIENCIAS DE LA INGENIERÍA
CURSO: MANEJO E IMPLEMENTACIÓN DE ARCHIVOS



MANUAL TÉCNICO
MINI ERP

ESTUDIANTE **CARNÉ:**
Cristian Alejandro Roldán López **202147280**

QUETZALTENANGO, 22 DE SEPTIEMBRE 2,026

Tecnologías usadas.

Lenguaje del servidor:
Java 21

Framework del servidor:
Spring Boot 4 (Spring Web, Spring Data JPA, Spring Security)

Framework del cliente:
Angular 21 (SPA, consumo de la API vía HttpClient)

Estilos del cliente:
TailwindCSS

Base de datos:
PostgreSQL

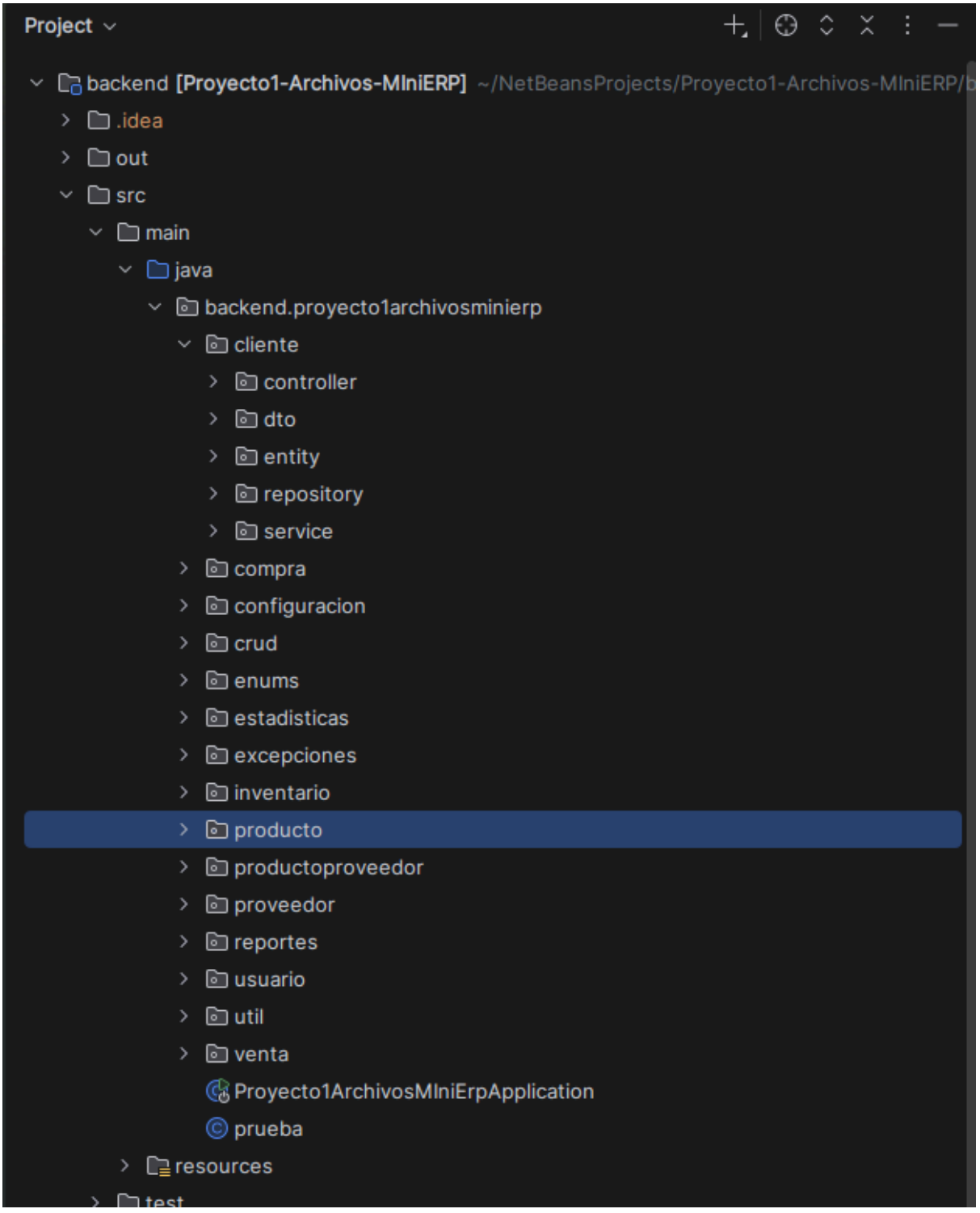
Control de migraciones:
Flyway

Autenticación:
JSON Web Tokens (JWT)

Evitar código repetitivo:
Lombok (@RequiredArgsConstructor, getters/setters)

Build / gestor de dependencias:
Maven

Estructura del proyecto.



El paquete raíz del servidor es backend.proyecto1archivosminierp. Cada módulo de negocio sigue la misma organización en capas:

- **controller**: expone los endpoints REST del módulo y delega en el service correspondiente
- **service:** contiene la lógica de negocio y las reglas de validación
- **repository:** interfaces que extienden JpaRepository y agregan consultas específicas
- **entity:** clases anotadas con JPA que representan las tablas de la base de datos.
- **dto:** objetos de transferencia de datos, separados en RequestDTO (datos de entrada) y ResponseDTO (datos de salida), además de DTOs especializados para reportes.

Esta separación evita exponer las entidades JPA directamente en la API y permite controlar con precisión qué información entra y sale del sistema.

Interfaz genérica para realizar un CRUD en todos los módulos

```
public interface Crud<REQUEST, RESPONSE, ID> {

    RESPONSE create(REQUEST request);

    RESPONSE update(ID id, REQUEST request);

    RESPONSE getById(ID id);

    List<RESPONSE> getAll();

    void delete(ID id);
}
```

Manejo de excepciones.

- **DatosNoEncontrados** : se lanza cuando un recurso solicitado (producto, cliente, proveedor, etc.) no existe.

- **DatosInvalidos**: se lanza cuando los datos de entrada no cumplen las reglas de validación (cantidades o montos inválidos, campos obligatorios vacíos).

- **OperacionInvalida**: se lanza cuando una operación de negocio no puede completarse en el estado actual del sistema (por ejemplo, una venta que dejaría existencias negativas).

El acceso al sistema se controla mediante autenticación basada en JSON Web Tokens:

• **AuthService:** genera y valida los tokens JWT (generateToken, extractUsername, extractClaim, isTokenValid, isTokenExpired) firmados con una clave secreta (jwt.secret) y con un tiempo de expiración configurable (jwt.expiration).

• **SecurityConfig:** configura la cadena de filtros de Spring Security, el CORS y el codificador de contraseñas (PasswordEncoder), e integra el filtro JWT en la cadena de seguridad.

• **UsuarioService / LoginService:** validan las credenciales (username y contraseña) contra la base de datos y devuelven el token junto con el rol del usuario autenticado.

Cada usuario está asociado a un Rol (Administración, Compras, Inventario o Ventas), lo que permite restringir el acceso a los endpoints según el área correspondiente, tal como lo exige el enunciado del proyecto.

Módulo de reportes.

Cada área de negocio cuenta con su propio servicio de reportes, apoyado en repositorios con consultas agregadas (JPQL/SQL nativo) sobre los datos históricos:

- **ReporteProductoRepository / ReporteInventarioService:** productos más vendidos, productos con menor existencia, historial de movimientos por producto.

- **ReporteCompraRepository / ReporteCompraService:** compras por rango de fechas, top de proveedores, productos más adquiridos.

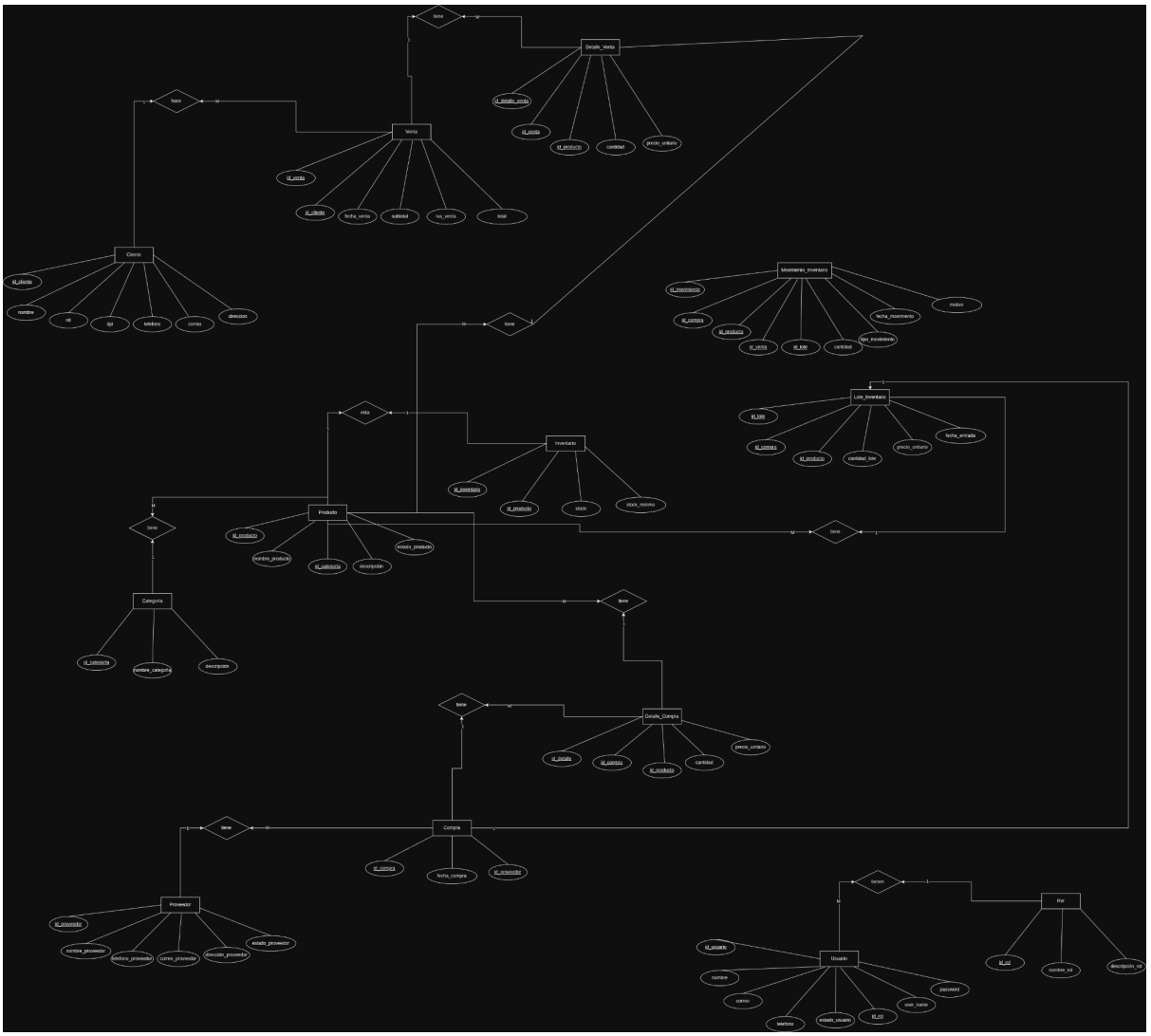
- **ReporteVentaRepository /ReporteVentaService:** ventas por rango de fechas, top de clientes, top de productos por ingresos, resumen de ventas por período (ResumenVentaDTO).

- **EstadisticaController:** expone indicadores generales del sistema combinando varios repositorios a la vez.

- **LogService:** reporte de auditoría de interacciones de usuarios.

Carga de datos iniciales.
La clase de arranque de la aplicación (Proyecto1ArchivosMiniErpApplication) convive con un componente independiente (identificado en el diagrama de clases como el poblador de datos) que ejecuta la carga inicial exigida por el enunciado: usuarios por área, productos, proveedores, clientes, compras y ventas de prueba, insertados mediante las migraciones de Flyway.

Diagrama E/R



Para mejor visualización
https://drive.google.com/file/d/1m2z90j8jbiQ1Xyh6zjmPSNHBRcC6rM6s/view?usp=sharing

Diagrama de casos de uso y de secuencia
https://drive.google.com/file/d/1FBPrGoK-hlW-Df-1dAlyZEF6MbWd5zwm/view?usp=sharing